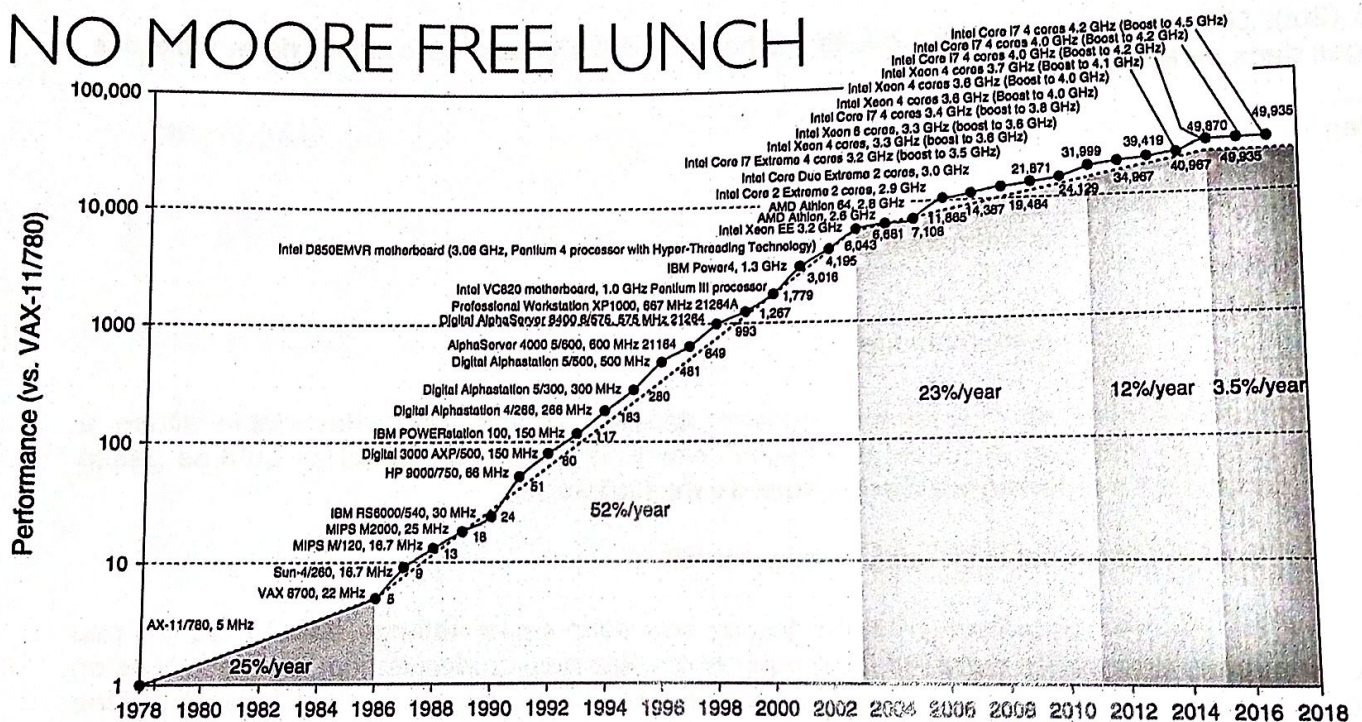


Examen de Systèmes – 1e session
11 décembre 2020, 10h20–12h30
L3 MIAGE

Les documents sont autorisés: notes de cours, TD, TP, mais uniquement sous forme papier. Les documents électroniques ne sont pas autorisés.
 Les réponses aux questions doivent être courtes et précises.

Q1 (2pt): Le graphique ci-dessous a été posté sur Twitter par un internaute qui a ajouté la légende: "Est-ce que vous vous souvenez de la Loi de Moore? Elle est morte depuis longtemps". D'un autre côté, d'autres spécialistes affirment que cette loi continue de s'appliquer aujourd'hui. Qui a raison, qui a tort? Expliquez votre réponse.

NO MOORE FREE LUNCH



Hennessy, John L., and David A. Patterson. *Computer architecture: a quantitative approach*. 6th ed, 2017

Q2 (3pt): On veut construire un nouveau circuit logique avec 6 entrées et 1 sortie. Le résultat en sortie vaut 1 si le nombre d'entrées qui valent 1 est un multiple de 3. Il vaut 0 sinon. **Dessinez le circuit logique et expliquez son fonctionnement en quelques phrases.**

Q3 (3pt): Que réalise le microprogramme suivant? **Quelle instruction assembleur implémente-t-il?**

xxx1: MAR = SP
 xxx2: H = TOS
 xxx3: MDR = TOS = TOS + H; wr; goto Main1

Q4 (3pt): Traduisez le programme suivant en assembleur IJVM (l'assembleur de la Mic1). Vous pouvez ignorer la déclaration de la variable `l`. La fonction `afficher()` a été déclarée préalablement.

```
int l;
```

```
if (l == 3) { afficher(l); }
else       { afficher(0); }
```

Q5 (2pt): Deux processus utilisent simultanément une variable partagée.

- Le premier processus modifie régulièrement la variable partagée dans laquelle il stocke le résultat de calculs qu'il effectue.
- Le deuxième processus lit une fois par seconde cette variable partagée et affiche sa valeur à l'écran.

Ces deux processus peuvent-ils être en situation de "race condition"? Expliquez votre réponse. Des mesures sont-elles nécessaires pour assurer une exécution correcte, et lesquelles?

Q8 (2pt): Une machine utilise des pages mémoire de 1024 octets. Le contenu de la table des pages dans un MMU est le suivant:

Page virtuelle	Page physique
0	1
1	Pas en mémoire vive
2	3
3	Pas en mémoire vive

Le processus effectue maintenant une écriture dans la variable `nouvellevariable` située à l'adresse virtuelle 1024, au début de la page virtuelle 1. Expliquez en détail ce qu'il se passe ensuite de façon à permettre à cette écriture de s'effectuer.

Q9 (1pt): Que fait l'appel système `kill()`? A quoi sert-il?

Q10 (2pt): Le programme suivant produit-il peu ou beaucoup de **localité spatiale**? Produit-il peu ou beaucoup de **localité temporelle**? Quel pourrait être (approximativement) le taux de succès en cache L1 (cache hit rate) lorsqu'il s'exécute sur une machine avec un cache L1 de 1ko? Une variable de type `int` utilise 4 octets en mémoire.

```
int matrice[1000][1000];
for (int i=0; i<1000; i++) {
    for (int j=0; j<1000; j++) {
        matrice[i][j] = i * j;
    }
}
```

Q10 (2pt): Un ex-étudiant de L3 MAGE a décidé de développer son propre système de fichiers. Dans ce système, les méta-données de chaque fichier ne sont pas stockées dans l'inode mais au début du premier bloc de données : les octets 0-31 du premier bloc de données contiennent l'identifiant du propriétaire du fichier, les droits d'accès, la taille du fichier, la date de dernière modification, etc. Le contenu du fichier proprement dit commence à partir de l'octet 32. Ainsi, affirme-t-il, même si on perd l'inode d'un fichier, il devient impossible de perdre les méta-données associées au fichier et on peut les reconstituer facilement lors d'une réparation du système de fichiers. Pensez-vous que ce soit une bonne idée ? Expliquez quels sont les avantages et/ou les inconvénients de cette organisation.

Examen 2018

Q₁: - le multiprocesseur transmet les données tandis que le décodeur interprète les données codées

Q₂:) Sujet Examen 2019

Q₂!)

Code Java

```
for (i=0; i<3; i++)  
{  
    affiche(i);  
}
```

BIPUSH 3

ISTORE i

Boucle: IFEQ FIN

INVOKEVIRTUAL Afficher

iload i

BIPUSH 1

ISUB

ISTORE i

GOTO Boucle

FIN:

Q₃:

Q₄: L'appel système kill arrête le processus.

2016 - 2017
2017 - 2018
2018 - 2019
2019 - 2020
2020 - 2021

Loi de Moore.

Q1 (1pt) : Depuis 2005 environ les fréquences des CPUs ont cessé d'augmenter. Quelle est la raison qui justifie ce phénomène ? N'est-ce pas une contradiction de la loi de Moore ?

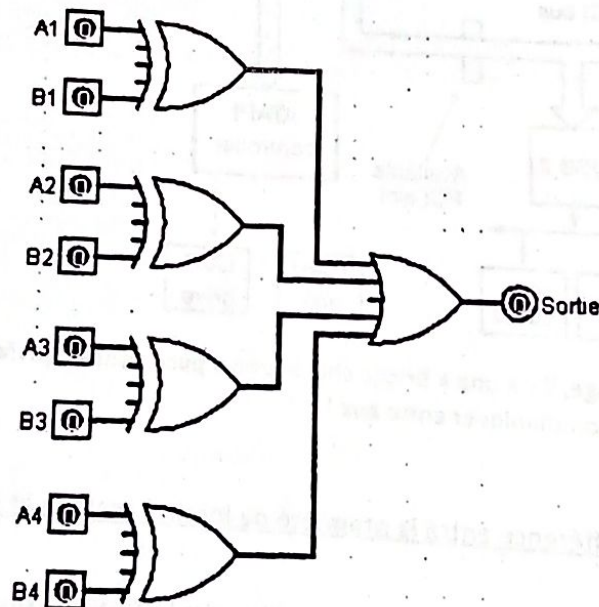
On ne peut pas réduire la taille d'un transistor à l'infini, il ne peut être plus petit qu'un atome, et il faut laisser le temps à l'information de se propager (à 3GHz, la lumière parcourt 10cm en un cycle d'horloge) on ne peut donc pas augmenter la fréquence des CPU à l'infini non plus !

Des questions de consommation d'énergie et de refroidissement se pose aussi lorsque l'on augmente la fréquence des CPUs.

La loi de Moore ne concerne pas les performances mais le nombre de transistor sur une puce, cette augmentation passe aujourd'hui sur les processeurs multicœurs. Bien que l'on sache faire des cœurs à 6GHz, deux cœurs à 3GHz consomment moins qu'un à 6.

Cela n'enter donc pas en contradiction avec la loi de Moore.

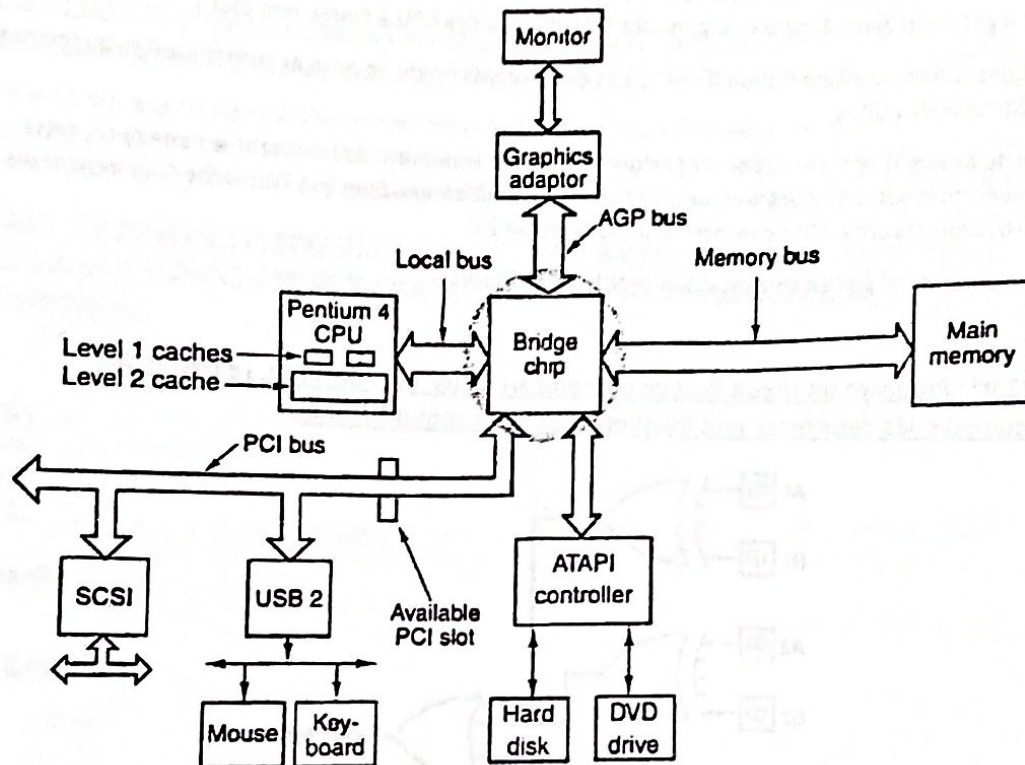
Q2 (2pt) : Proposez un circuit logique qui compare deux mots de 4 bits. Le circuit doit produire 0 si les deux mots sont identiques, et 1 s'ils sont différents.



Q3 (1pt) : Les premiers ordinateurs utilisaient un seul bus et connectaient tous les périphériques sur ce même bus. Pourquoi était-ce une mauvaise idée ?

Cela n'était pas une bonne idée car peut équitale pour les périphériques : si le périphérique le plus proche de l'arbitre demande souvent à faire une requête, il aura plus souvent la parole que le périphérique le plus loin !

Q4 (2pt) : Les ordinateurs modernes disposent de plusieurs bus connectés les uns aux autres. Par exemple, dans la figure ci-dessous, le bus USB2 est connecté sur le bus PCI. Cependant, différents bus utilisent différents formats de message et protocoles de communication. De quelle façon pensez-vous que ces bus sont reliés entre eux de façon à ce que le CPU puisse communiquer avec le clavier par exemple ?



Comme on le voit sur l'image, il y a une « Bridge chip » (une « puce pont ») qui fait le lien entre tout les bus et leur permet de communiquer entre eux !

Q5 (1pt) : Expliquez la différence entre la propriété de localité spatiale et la propriété de localité temporelle.

La propriété de localité spatiale implique que l'on va utiliser des instructions situées dans une zone mémoire proche des autres instructions accédé récemment (exemple : un tableau) alors que la propriété de localité temporelle implique que l'on va utiliser des instructions déjà utilisé dans un passé plutôt proche (exemple : accéder au sommet de la pile).

Q6 (2pt) : On veut concevoir un ordinateur qui exécutera essentiellement des programmes avec beaucoup de localité temporelle mais très peu de localité spatiale. Quelle sera une bonne valeur pour la taille des lignes de cache mémoire ? Expliquez votre réponse.

La taille des lignes de cache mémoire ne devrait pas être trop grande, de quoi stocker une instruction. Une petite taille de ligne permet d'avoir plus de lignes au total dans le cache, cela permet donc de garder plus d'instruction utilisées dans le passé en cache, la ou des grandes lignes de cache permettrait de charger beaucoup d'informations après une instruction (comme la suite d'un tableau par exemple), mais réduirait le nombre de ligne disponible en cache (donc le nombre d'instructions disponible en cache).

Q7 (1pt) : Les processeurs modernes utilisent des pipelines avec 7 à 10 étapes. Expliquez la raison pour laquelle rajouter davantage d'étapes n'est pas forcément une bonne idée.

On ne peut pas augmenter à l'infini le nombre d'étapes : plus il y a d'étape, plus il y a d'opérations (exemple pour un if : en attendant la réponse du if, on effectue les opérations qui suivent en supposant que le if est true, si la réponse est false, on doit refaire toutes les opérations déjà faites).

Q8 (2pt) : Traduisez le morceau de programme Java ci-dessous en byte-code JVM. Vous pouvez ignorer la déclaration de la variable i. La méthode afficher() a été déclarée préalablement.

```
int i;  
(...)  
if (i>0) {  
    i = i + 1;  
} else {  
    i = i - 2;  
}  
afficher(i);
```

Code JVM:

```
ILOAD 1  
IFLT L1  
BIPUSH 1  
IADD  
STORE i  
ILOAD  
L1: BIPUSH 2  
ISUB  
STORE i  
ILOAD i  
INVOKEVIRTUAL AFFICHER
```

Q9 (1pt) : Les deux programmes ci-dessous s'exécutent simultanément sur la même machine. Y a-t-il un risque de "race condition" ? Expliquez votre réponse.

Programme 1 (écrit en Java):	Programme 2 (écrit en C):
<pre>int userid = fichier.read(); if (userid <= 0) throw new UnknownUser(); authenticateUser(userid);</pre>	<pre>int userid, ok; for (userid=0; userid<1000; userid++) { ok = verifierUtilisateur(userid); }</pre>

Il n'y a pas de risque de « race condition » car les variables `userid` des deux programmes sont local et ne peuvent être accéder en dehors d'eux même. Ces variables on le même nom mais le programme en Java ignore même jusqu'à l'existence de la variable du programme en C (et vise vers ça).

Q10 (2pt) : Une machine utilise des pages mémoire de 1024 octets. Le contenu de la table des pages dans un MMU est le suivant :

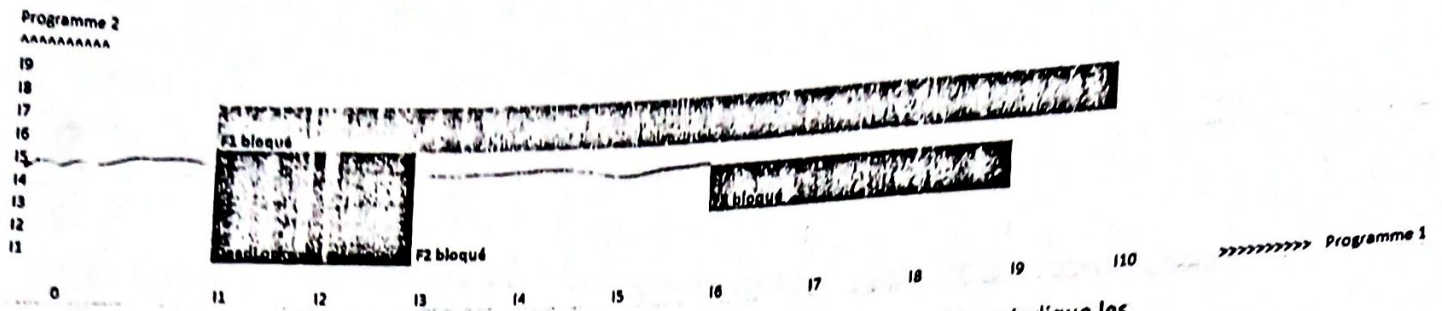
Page virtuelle	Page physique
0	2
1	Pas en mémoire vive
2	0
3	Pas en mémoire vive

Quelles sont les adresses physiques correspondant aux adresses virtuelles 1000, 1025 et 2500 ?

Q11 (2pt) : Deux processus manipulent les mêmes fichiers F1, F2 et F3. Pour éviter les "race conditions" ils verrouillent ces fichiers avant de s'en servir :

Processus 1:	Processus 2:
<pre>Verrouiller F1; Lire depuis le clavier et écrire dans F1; Verrouiller F2; Lire le contenu de F2, l'écrire dans F1; Déverrouiller F2; Verrouiller F3; Effacer le contenu de F3; Copier le contenu de F1 dans F3; Déverrouiller F3; Déverrouiller F1;</pre>	<pre>Verrouiller F2; Lire le contenu de F2; Verrouiller F3; Ecrire le résultat de la lecture dans F3; Déverrouiller F3; Verrouiller F1; Copier le contenu de F2 dans F1; Déverrouiller F1; Déverrouiller F2;</pre>

Ces deux processus peuvent-ils entrer en deadlock (interblocage) ? Utilisez l'algorithme du banquier pour indiquer comment garantir une exécution sans deadlock.



Ces deux processus peuvent entrer en deadlock, la zone rouge sur le graphique indique les instructions à bloquer pour ne pas entrer en deadlock.

Q12 (2pt) : Un ex-étudiant de L3 MAGE a décidé de développer son propre système de fichiers. Dans ce système, les métadonnées de chaque fichier ne sont pas stockées dans l'inode mais au début du premier bloc de données : les octets 0-31 du premier bloc de données contiennent l'identifiant du propriétaire du fichier, les droits d'accès, la taille du fichier, la date de dernière modification, etc. Le contenu du fichier proprement dit commence à partir de l'octet 32. Ainsi, affirme-t-il, même si on perd l'inode d'un fichier, il devient impossible de perdre les métadonnées associées au fichier et on peut les reconstituer facilement lors d'une réparation du système de fichiers. Pensez-vous que ce soit une bonne idée ? Expliquez quels sont les avantages et/ou les inconvénients de cette organisation.

Si les métadonnées sont stockées et dans l'inode et dans les premiers octets du fichier, cela fait doublons et peut engendrer beaucoup de perte d'espace s'il on a beaucoup de fichier. Mais cela permet en effet une sécurité en plus en cas de problème au niveau de l'inode, cela dépend donc de ce que l'on souhaite en faire : un système plus optimisé dans la gestion de l'espace ne devrait pas l'utiliser, mais s'il on cherche à faire un système sécurisé qui n'a pas de limite d'espace pour les données, cela peut être utile.

Q13 (1pt) : Dans le système de fichier FAT16 une entrée dans un répertoire utilise 32 octets, ce qui permet de stocker jusqu'à 16 entrées dans un bloc de 512 octets. Comment le système gère-t-il des répertoires qui contiennent plus que 16 fichiers ?